

Conteneurisation, Orchestration et Pipeline CI/CD

Architecture distribuée d'un agent de collecte de métriques système et API REST FastAPI

Dépôt GitHub : [rosenatachambourou-prog/metrics-agent-devops](https://github.com/rosenatachambourou-prog/metrics-agent-devops)

Docker Hub : [serge000/metrics-agent:latest](https://hub.docker.com/r/serge000/metrics-agent:latest)

Cloud Live (Railway) : metrics-agent.up.railway.app

01 / COMPOSITION DE L'ÉQUIPE & ENVIRONNEMENTS MATÉRIELS

COLLABORATEUR	RESPONSABILITÉS & DOMAINE D'INTERVENTION	MACHINE PHYSIQUE DE TRAVAIL
MAVOUNGOU Serge Murlain	Conteneurisation Docker, orchestration Compose, pipeline CI initial, push Docker Hub	Apple MacBook Air M4, 32 Go (arm64)
MBOUROU GYAMERAH Rose Natacha Liyane	Gestion du dépôt GitHub, secrets de déploiement, relecture PRs, validation amd64	HP EliteBook 840, 8 Go, Win 10 (amd64)
MAKOSSO Loïck Esdras	Audit de conformité, refonte pipeline 3 jobs (Fail-Fast), sécurité, preuves, Cloud Railway	Apple MacBook Air M1, 8 Go (arm64)

02 / ARCHITECTURE APPLICATIVE & TOPOLOGIE RÉSEAU

L'application orchestre deux services distincts au sein d'un même réseau virtuel privé metrics-net. L'agent échantillonne l'état physique du serveur via la bibliothèque psutil toutes les 5 secondes et pousse ces données vers l'API FastAPI. La communication inter-conteneurs s'appuie strictement sur la résolution DNS interne Docker (http://api:8000/metrics), sans IP statique.



03 / CONTENEURISATION OPTIMISÉE & SÉCURISÉE (DOCKER)

Construction multi-étape (multi-stage) : L'image s'appuie sur une base Debian python:3.12-slim pour garantir la compatibilité glibc nécessaire à psutil. L'étage intermédiaire (builder) crée un environnement virtuel /opt/venv et compile les dépendances. L'étage final (runtime) recopie uniquement cet environnement et le code métier, éliminant totalement les compilateurs et caches pip. L'image finale atteint une taille ultra-légère de 283 Mo sur disque et 61 Mo compressés au registre Docker Hub.

Exécution sans privilèges root : Un compte système dédié app (UID/GID 10001:10001) est créé sans privilèges sudo ni shell interactif (/usr/sbin/nologin). L'instruction USER 10001:10001 isole le processus contre toute élévation de privilèges. La sonde HEALTHCHECK est implémentée nativement via Python (urllib.request), évitant l'injection superflue du binaire curl.

04 / ORCHESTRATION DÉCLARATIVE MULTI-SERVICES (COMPOSE)

L'architecture Compose est organisée pour séparer nettement les contraintes de production et de développement local :

- Ordonnancement et résilience : L'agent déclare `depends_on: api: condition: service_healthy`. Ce garde-fou empêche tout démarrage anticipé du collecteur tant que le serveur web n'a pas validé avec succès sa propre sonde `/health`.
- Validation stricte des variables d'environnement : Le fichier `docker-compose.yaml` utilise l'expansion `${DOCKERHUB_USERNAME:?..}`. En cas d'omission du fichier `.env`, Compose lève immédiatement une erreur explicite sans risquer de repli non sécurisé.
- Surcouche de développement transparent (override) : Le fichier `docker-compose.override.yml` monte le code source en volume local (`./app:/app/app`) et cible `Dockerfile.dev` avec l'option `uvicorn --reload` pour offrir le rechargement à chaud sans aucune altération de la pile de production.

05 / PIPELINE D'INTÉGRATION ET DÉPLOIEMENT CONTINU (CI/CD)

Initialement monolithique, le workflow GitHub Actions (`.github/workflows/ci-cd.yml`) a été refondu en 3 jobs modulaires et séquentiels pour éliminer le gaspillage de ressources et garantir la sécurité des publications :

JOB & RÔLE	CONDITIONS & DÉPENDANCES	MÉCANISMES CLÉS & GARANTIES DE SÉCURITÉ
1. test (Tests unitaires)	Déclenché sur PR et main Sans dépendance	Exécution immédiate de <code>pytest -q</code> sur Python 3.12 hôte avec <code>cache pip</code> . Principe de Fail-Fast : valide le code en ~11 secondes ; interrompt net le pipeline en cas de régression sans déclencher de build Docker.
2. container-test (Validation conteneur)	Déclenché sur PR et main <code>needs: test</code>	Construction locale rapide via <code>Buildx</code> (cache GitHub Actions). Lancement du conteneur en tâche de fond et polling de la sonde <code>/health</code> . Nettoyage déterministe garanti par la directive <code>trap 'docker rm -f api-ci' EXIT</code> .
3. publish (Publication Docker Hub)	Branche main uniquement <code>needs: container-test</code>	Garde-fou : aucune PR ne peut pousser d'image. Initialisation ordonnée de QEMU puis <code>Buildx</code> . Publication multi-plateforme simultanée pour <code>linux/amd64</code> et <code>linux/arm64</code> avec double étiquetage <code>latest</code> et <code>sha-commit</code> .

06 / ANALYSE DE SÉCURITÉ : LE PRINCIPE DE MOINDRE PRIVILÈGE

Arbitrage technique sur les accès Docker Hub : Lors de l'automatisation de la documentation du registre via l'action `peter-evans/dockerhub-description`, une erreur HTTP 403 Forbidden est survenue. L'API Docker Hub exige en effet un jeton d'accès doté des privilèges Administrateur (Admin) pour modifier les métadonnées d'un dépôt.

Décision d'ingénierie DevOps : Conformément au principe de moindre privilège (*Least Privilege*), nous avons refusé d'accorder des droits administrateur globaux au pipeline pour une tâche documentaire non critique. Le jeton d'accès GitHub Actions a été maintenu strictement restreint aux droits Lecture & Écriture d'images (*Read & Write*). La tâche a été sécurisée avec `continue-on-error: true`, et la documentation complète est conservée sous contrôle de version dans le fichier `DOCKERHUB.md`.

07 / VALIDATION EN PRODUCTION CLOUD (RAILWAY — BONUS)

Pour concrétiser la livraison logicielle au-delà des environnements locaux, l'image Docker a été déployée sur la plateforme Cloud Railway. L'infrastructure assure la haute disponibilité en HTTPS avec terminaison TLS managée :

- Disponibilité vérifiable : `curl -s https://metrics-agent.up.railway.app/health` répond `{"status": "ok"}`.
- Flux temps réel : `curl -s https://metrics-agent.up.railway.app/metrics` restitue les séries temporelles générées par le démon.
- Documentation OpenAPI interactive : Interface Swagger disponible en accès public sur `https://metrics-agent.up.railway.app/docs`.

08 / GOUVERNANCE GIT & TRAÇABILITÉ EXHAUSTIVE DES CONTRIBUTIONS

L'équipe a appliqué une discipline stricte basée sur GitHub Flow. La branche main est restée protégée : aucune modification directe n'y a été portée. Chaque tâche a suivi le cycle : Branche thématique -> Validation automatisée CI/CD -> Relecture -> Fusion sur main. Voici le tableau de traçabilité complet couvrant les 21 Pull Requests du projet :

LIVRABLE / MISSION RÉALISÉE	AUTEUR	RELECTURE	RÉFÉRENCE GIT
Séparation des dépendances (requirements-dev), pytest.ini	Serge	Rose	Commit socle
Fichier .dockerignore et Dockerfile multi-stage de production	Serge	Rose	Pull Request #1
Dockerfile.dev pour le rechargement à chaud en développement	Serge	Rose	Pull Request #2
Orchestration de production docker-compose.yaml	Serge	Rose	Pull Request #3
Surcouche docker-compose.override.yml et gabarit .env.example	Serge	Rose	Pull Request #4
Pipeline initial CI/CD GitHub Actions (build et push)	Serge	Rose	Pull Request #6
Rédaction initiale de la documentation README.md	Serge	Rose	Pull Request #7
Initialisation dépôt GitHub, secrets de déploiement et branches	Rose	—	Admin Dépôt
Validation du tirage et fonctionnement sur architecture amd64	Rose	—	Recette Windows
Sécurisation secrets CI/CD Docker Hub et nettoyage garanti trap	Loïck	Rose	Pull Request #11
Neutralisation identifiants et durcissement .gitignore / .env	Loïck	Rose	Pull Request #12
Robustesse et gestion d'erreurs Compose ({DOCKERHUB_USERNAME:...})	Loïck	Rose	Pull Request #13
Documentation modes standalone Dockerfile.dev et prérequis	Loïck	Rose	Pull Request #14
Conception et intégration des 8 captures de preuves d'exécution (SVG)	Loïck	Rose	Pull Request #15
Automatisation de la documentation Docker Hub via DOCKERHUB.md	Loïck	Rose	Pull Request #16
Déploiement Cloud sur Railway (bonus) et intégration des endpoints	Loïck	Rose	Pull Request #17
Résilience CI et montée de version dockerhub-description@v5	Loïck	Rose	Pull Request #18
Refonte modulaire du pipeline en 3 jobs ordonnés (Fail-Fast)	Loïck	Rose	Pull Request #19
Formalisation sécurité du moindre privilège (droits admin Docker Hub)	Loïck	Rose	Pull Request #20
Actualisation des contributions d'équipe et spécifications machines	Loïck	Rose	Pull Request #21

09 / BILAN TECHNIQUE & CONCLUSION

Ce projet a permis d'éprouver et d'appliquer avec rigueur les standards industriels du DevOps contemporain : sobriété et étanchéité des images conteneurisées (construction multi-stage, compte non-root 10001:10001), déterminisme de l'orchestration locale (réseau privé pont, dépendances ordonnées via healthcheck), vitesse et fiabilité de l'intégration continue (pipeline en 3 jobs avec retour de test en 11 secondes), portabilité universelle par compilation croisée native (amd64 et arm64 vérifiées sur Apple Silicon et Windows), prudence opérationnelle par respect du moindre privilège, et enfin aboutissement complet par déploiement Cloud mondial sur Railway. L'ensemble du cahier des charges et du barème d'évaluation a été intégralement validé par des éléments de preuve auditables et vérifiables.